NYCU DL LAB1 Report

313551127 王翔

1. Introduction:

In this lab, we implemented a neural network using Numpy that can classify linear and XOR datasets separately. We developed forward and backward functions to achieve prediction and gradient descent. During training, we can set appropriate hyperparameters, such as the number and dimensions of hidden layers, learning rate, etc. Additionally, we can choose the activation function and optimizer to aid in training different datasets. This allows for a better understanding of how different models and hyperparameters affect the performance on the two datasets.

2. Experiment setups:

A. Sigmoid functions:

```python
def Sigmoid(x, derivative = False):
    if not derivative:
        return 1.0/ (1.0 + np.exp(-x))
    else:
        return x * (1 - x)
```

B. Neural network:

When building the model, you can set the layers to be either convolution or fully connected, specify the number of hidden layers, the size of hidden dimensions, the activation function, the optimizer, etc. Each layer is constructed from previously defined layer classes. The forward and backward operations are initiated by model.forward and model.backward, sequentially calling each layer's forward and backward methods. Training is conducted using model.train. Finally, the model.show_result (not shown in the image due to space constraints) outputs the required results and images.

```
class Model:
    def __init__(self, optim, layer_type, input_dim = 2, hidden_layer_num = 2, hidden_dim = 4,
                 output_dim = 1, lr = 0.01, activation = "Sigmoid", alpha = alpha_for_ELU):
        self.layers = []
        self.losses = []
        #collect every layers in order
        self.input_dim = input_dim
        self.output_dim = output_dim
        self.hidden_layer_num = hidden_layer_num
        self.hidden_dim = hidden_dim
        self.lr = lr
        self.activation = activation
        self.alpha = alpha
        self.optim = optim

        if optim.name == "Adam":
            self.optims = []
            for i in range(self.hidden_layer_num + 2):
                self.optims.append(Adam_optimizer())
        elif optim.name == "RMSprop":
            self.optims = []
            for i in range(self.hidden_layer_num + 2):
                self.optims.append(RMSprop_optimizer())

        if layer_type == "FC":
            self.input_layer = FC_Layer(input_dim = self.input_dim, output_dim = self.hidden_dim,
                                        activation = self.activation, alpha = self.alpha)
            self.output_layer = FC_Layer(input_dim = self.hidden_dim, output_dim = self.output_dim,
                                         activation = self.activation, alpha = self.alpha)
            self.layers.append(self.input_layer)
            for i in range(self.hidden_layer_num):
                self.layers.append(FC_Layer(input_dim = self.hidden_dim, output_dim = self.hidden_dim,
                                            activation = self.activation, alpha = self.alpha))

        elif layer_type == "Conv1d":
            #self.input_layer = Conv1d_Layer(input_dim = self.input_dim, filter_size = filter_size, stride = stride,
            #                                padding = padding, activation = self.activation, alpha = self.alpha)
            self.input_layer = FC_Layer(input_dim = self.input_dim, output_dim = self.hidden_dim,
                                        activation = self.activation, alpha = self.alpha)
            self.layers.append(self.input_layer)
            #out = (self.input_dim - filter_size + 2 * padding) // stride + 1
            out = self.hidden_dim
            for i in range(self.hidden_layer_num):
                self.layers.append(Conv1d_Layer(input_dim = out, filter_size = filter_size, stride = stride,
                                                padding = padding, activation = self.activation, alpha = self.alpha))
                out = (out - filter_size + 2 * padding) // stride + 1

            self.output_layer = FC_Layer(input_dim = out, output_dim = self.output_dim,
                                         activation = self.activation, alpha = self.alpha)
        self.layers.append(self.output_layer)



    def forward(self, X):
        A = X
        for layer in self.layers:
            A = layer.forward(A)
        return A

    def backward(self, y_true):
        layers_num = len(self.layers)
        y_pred = self.layers[-1].A
        #prediction is equal to A in the last layer

        dA = MSELoss(y_true, y_pred, derivative = True)
        #apply derivative MSELoss
        grad = 0
        for i in reversed(range(layers_num)):
        # backward so in reverse order
            dA = self.layers[i].backward(dA)
            grad += np.mean(self.layers[i].dW)
            #update dA and pass it to prev layer
        #print(f"gradient in layer {i} = {grad}")
    def update_parameters(self, lr):
        for i, layer in enumerate(self.layers):
            if self.optim.name == "Adam":
                layer.update_parameters(lr,self.optims[i])
            elif self.optim.name == "RMSprop":
                layer.update_parameters(lr,self.optims[i])
            else:
                layer.update_parameters(lr, self.optim)

    def train(self, X, Y, epochs = 100000):
        for epoch in range(epochs):
            y_pred = self.forward(X)
            loss = MSELoss(Y, y_pred, derivative = False)
            self.losses.append(loss)
            #calculate loss with MSELoss
            self.backward(Y)
            #using loss to compute gradient for each layer from backward

            self.update_parameters(self.lr)
            #gradient descent

            if (epoch)% output_epoch == 0:
                print(f"epoch {epoch: <5} loss : {loss:<8.9f}")
```

C. Backpropagation:

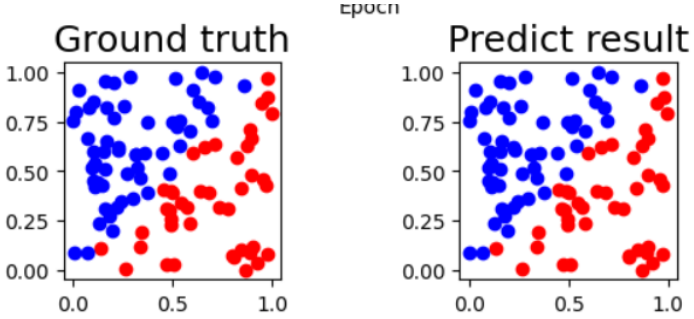
In each layer, we record its weights and biases for easy calculation and updates later on. The forward function calculates the values and passes them to the next layer through the activation function. The backward function takes the dA returned from the next layer, passes it through the derivative activation function, and then calculates dW and db. Finally, the update_parameters function uses the specified optimizer and learning rate to update the weights and biases using dW and db.

```python
class FC_Layer:
    def __init__(self, input_dim, output_dim, activation = "Sigmoid", alpha = 0.1):
        self.input_dim = input_dim
        self.output_dim = output_dim
        self.activation = activation
        self.alpha = alpha
        self.weight = np.random.randn(input_dim, output_dim) * np.sqrt(2 / self.input_dim)
        #He init
        self.bias = np.random.randn(1, output_dim) * 0.1
        #bias init to small value
        self.A_prev = None
        self.Z = None
        self.A = None
        self.dW = None
        self.db = None
        self.dA_prev = None

    def forward(self, A_prev):
        self.A_prev = A_prev
        self.Z = np.dot(A_prev, self.weight) + self.bias      #Z is intermediate, Z = W^T * A + b


        match self.activation:
            #compute A with Z pass through activation function
            case "Sigmoid":
                self.A = Sigmoid(self.Z, derivative = False)
            case "ReLU":
                self.A = ReLU(self.Z, derivative = False)
            case "tanh":
                self.A = tanh(self.Z, derivative = False)
            case "ELU":
                self.A = ELU(self.Z, self.alpha, derivative = False)
            case _:
                #for no activation function
                self.A = self.Z

        return self.A

    def backward(self, dA):
        #dA is from prev layer of backward(next layer of forward)

        match self.activation:
            #coeupte dZ, where dZ is gradient of this layer
            case "Sigmoid":
                dZ = dA * Sigmoid(self.A, derivative = True)
            case "ReLU":
                dZ = dA * ReLU(self.A, derivative = True)
            case "tanh":
                dZ = dA * tanh(self.A, derivative = True)
            case "ELU":
                dZ = dA * ELU(self.A, self.alpha, derivative = True)
            case _:
                #for no activation function
                dZ = dA

        self.dW = np.dot(self.A_prev.T, dZ)
        #calculate dW by using dZ (store during forward pass)and A_prev (from forward pass)
        self.db = np.sum(dZ, axis = 0, keepdims = True)
        #calculate db by using dZ only
        self.dA_prev = np.dot(dZ, self.weight.T)
        return self.dA_prev
        #dA_prev is dA of this layer, so pass it to next layer

            # more activation add here

    def update_parameters(self, lr, optim):
        match optim.name:
            case "SGD":
                self.weight, self.bias = optim.update(lr, self.weight, self.bias, self.dW, self.db)
            case "Adam":
                self.weight, self.bias = optim.update(lr, self.weight, self.bias, self.dW, self.db)
            case "RMSprop":
                self.weight, self.bias = optim.update(lr, self.weight, self.bias, self.dW, self.db)
```

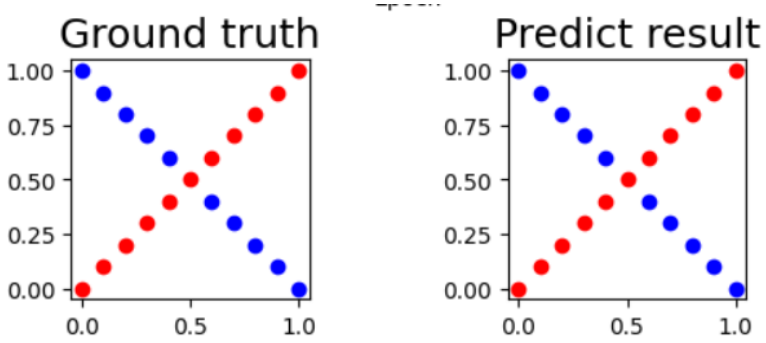
3. Results of your testing

A. Screenshot and comparison figure

Linear:



XOR:



B. Show the accuracy of your prediction

Hidden layers = 2, Hidden dim = 4, optimizer = SGD, activation = Sigmoid, lr = 0.1

Linear:

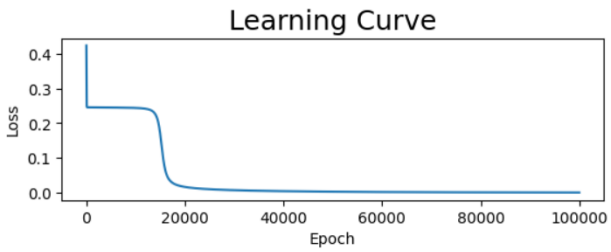
```
Iter 80  | Ground truth: 0 | Prediction: 0.000010008
Iter 81  | Ground truth: 0 | Prediction: 0.000010220
Iter 82  | Ground truth: 0 | Prediction: 0.000010149
Iter 83  | Ground truth: 1 | Prediction: 0.999830698
Iter 84  | Ground truth: 1 | Prediction: 0.997061656
Iter 85  | Ground truth: 0 | Prediction: 0.006152860
Iter 86  | Ground truth: 0 | Prediction: 0.024969842
Iter 87  | Ground truth: 1 | Prediction: 0.999918799
Iter 88  | Ground truth: 1 | Prediction: 0.987753871
Iter 89  | Ground truth: 0 | Prediction: 0.000011207
Iter 90  | Ground truth: 0 | Prediction: 0.000011399
Iter 91  | Ground truth: 0 | Prediction: 0.000010202
Iter 92  | Ground truth: 1 | Prediction: 0.999900322
Iter 93  | Ground truth: 1 | Prediction: 0.999918992
Iter 94  | Ground truth: 1 | Prediction: 0.999907668
Iter 95  | Ground truth: 0 | Prediction: 0.000010913
Iter 96  | Ground truth: 1 | Prediction: 0.999919035
Iter 97  | Ground truth: 1 | Prediction: 0.999917371
Iter 98  | Ground truth: 1 | Prediction: 0.999734982
Iter 99  | Ground truth: 0 | Prediction: 0.000017198
Iter 100 | Ground truth: 1 | Prediction: 0.999907877
loss = 0.0006841603224366842 accuracy = 100.0%
```

XOR:

```
Iter 1   | Ground truth: 0 | Prediction: 0.008584380
Iter 2   | Ground truth: 1 | Prediction: 0.998483767
Iter 3   | Ground truth: 0 | Prediction: 0.008597785
Iter 4   | Ground truth: 1 | Prediction: 0.998475161
Iter 5   | Ground truth: 0 | Prediction: 0.008613073
Iter 6   | Ground truth: 1 | Prediction: 0.998421553
Iter 7   | Ground truth: 0 | Prediction: 0.008630094
Iter 8   | Ground truth: 1 | Prediction: 0.998062706
Iter 9   | Ground truth: 0 | Prediction: 0.008648705
Iter 10  | Ground truth: 1 | Prediction: 0.979622988
Iter 11  | Ground truth: 0 | Prediction: 0.008668763
Iter 12  | Ground truth: 0 | Prediction: 0.008690130
Iter 13  | Ground truth: 1 | Prediction: 0.981485825
Iter 14  | Ground truth: 0 | Prediction: 0.008712669
Iter 15  | Ground truth: 1 | Prediction: 0.998297238
Iter 16  | Ground truth: 0 | Prediction: 0.008736245
Iter 17  | Ground truth: 1 | Prediction: 0.998597282
Iter 18  | Ground truth: 0 | Prediction: 0.008760726
Iter 19  | Ground truth: 1 | Prediction: 0.998649904
Iter 20  | Ground truth: 0 | Prediction: 0.008785977
Iter 21  | Ground truth: 1 | Prediction: 0.998669491
loss = 7.644024967282383e-05 accuracy = 100.0%
```

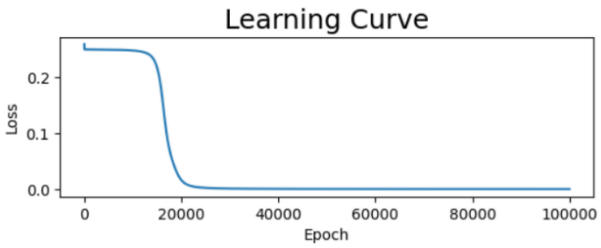
C. Learning curve (loss, epoch curve)

Linear:




```
epoch 0     loss : 0.423457164
epoch 5000  loss : 0.245124564
epoch 10000 loss : 0.243992487
epoch 15000 loss : 0.166829156
epoch 20000 loss : 0.016603404
epoch 25000 loss : 0.010063851
epoch 30000 loss : 0.007280455
epoch 35000 loss : 0.005619430
epoch 40000 loss : 0.004475734
epoch 45000 loss : 0.003625689
epoch 50000 loss : 0.002970246
epoch 55000 loss : 0.002456818
epoch 60000 loss : 0.002051828
epoch 65000 loss : 0.001730907
epoch 70000 loss : 0.001475263
epoch 75000 loss : 0.001270176
epoch 80000 loss : 0.001104195
epoch 85000 loss : 0.000968521
epoch 90000 loss : 0.000856462
epoch 95000 loss : 0.000762954
```

XOR:




```
epoch 0     loss : 0.258761904
epoch 5000  loss : 0.248996034
epoch 10000 loss : 0.247469870
epoch 15000 loss : 0.216999497
epoch 20000 loss : 0.016133426
epoch 25000 loss : 0.002158137
epoch 30000 loss : 0.000934262
epoch 35000 loss : 0.000565940
epoch 40000 loss : 0.000397276
epoch 45000 loss : 0.000302622
epoch 50000 loss : 0.000242734
epoch 55000 loss : 0.000201726
epoch 60000 loss : 0.000172029
epoch 65000 loss : 0.000149608
epoch 70000 loss : 0.000132124
epoch 75000 loss : 0.000118136
epoch 80000 loss : 0.000106708
epoch 85000 loss : 0.000097209
epoch 90000 loss : 0.000089195
epoch 95000 loss : 0.000082351
```

D. Anything you want to present

In this case, since the training dataset and the testing dataset are the same, overfitting is not a concern. Therefore, as long as we can consistently find the direction of gradient descent, increasing the number of training epochs can effectively improve accuracy and reduce loss.

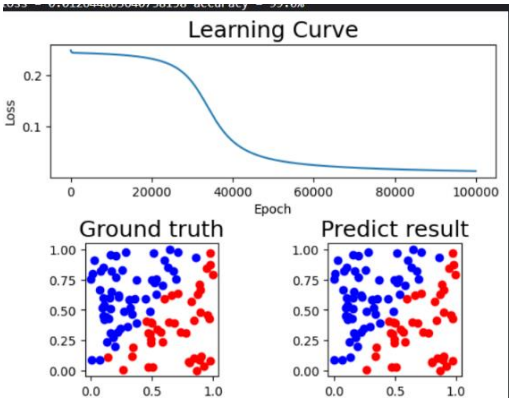
4. Discussion

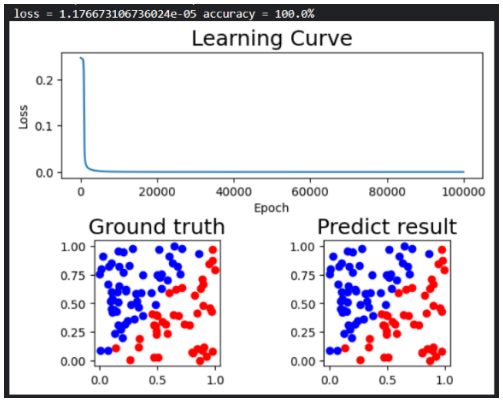
A. Try different learning rates

Hidden layers = 2, Hidden dim = 4, optimizer = SGD, activation = Sigmoid

lr = 0.01: lr = 1:

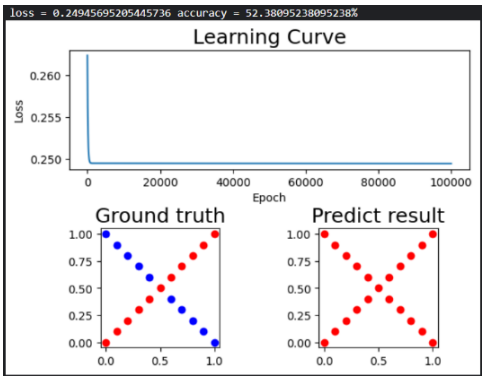
Linear:



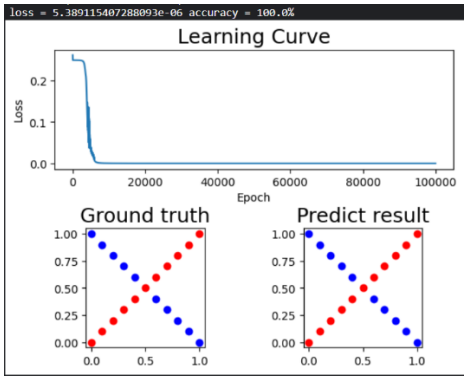


XOR:

lr = 0.01:



lr = 1:

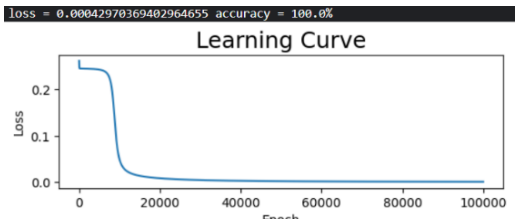


When using SGD as the optimizer, the learning rate does not adjust during training phase. Therefore, a smaller learning rate might not reduce the loss sufficiently, resulting in accuracy not reaching 100% in both Linear and XOR cases, especially for the XOR data, it was not successfully trained, it potentially requiring more epochs to achieve 100%. On the other hand, a larger learning rate can quickly decrease the loss in the early stages of training. However, we observed that the loss oscillates on the XOR data, which might be due to a learning rate that is too large, preventing the algorithm from successfully reaching a local minimum.

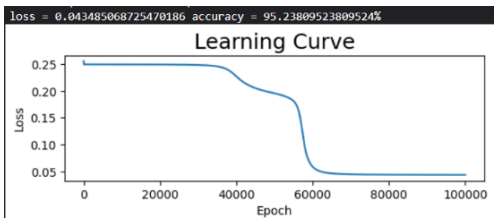
B. Try different numbers of hidden units

Hidden dim = 2:

Linear:



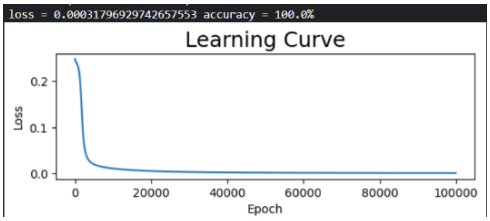
XOR:



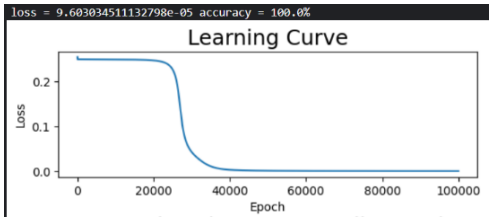
When the hidden dimension is set to 2, the linear data can be easily classified by finding a two-dimensional line. However, for the XOR data, the classification problem is more complex, requiring more epochs or larger learning rate to achieve 100% accuracy.

Hidden dim = 16:

Linear:



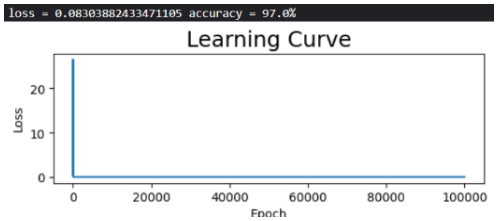
XOR:



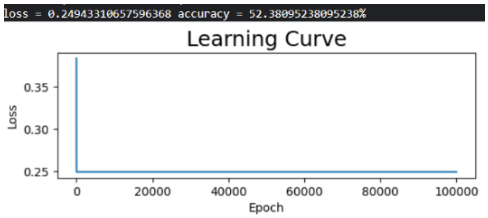
When the hidden dimension is set to 16, in linear case show a faster convergence rate. On the XOR data, it took more epochs to start reducing the loss. This is possibly because the more complex network structure is more sensitive to the initial values. This indicates that appropriately increasing the dimensions of the hidden layer is beneficial for our both cases. However, we need to pay more attention to the issue of initial values.

C. Try without activation functions

Linear:



XOR:

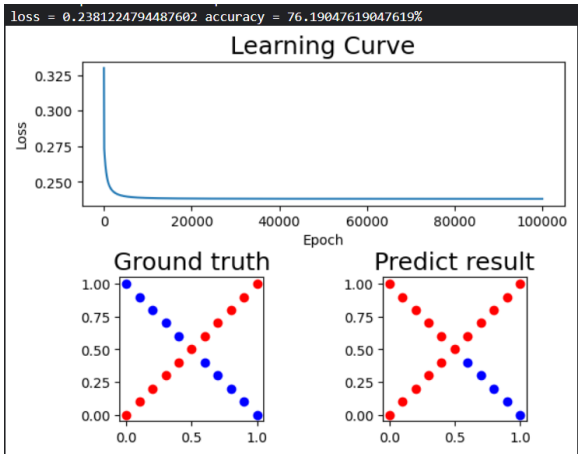


Without an activation function, the neural network loses its non-linear capabilities. Although the accuracy on the linear data is still high, it does not reach 100%. For the XOR data, the training fails because XOR cannot be classified using a linear approach.

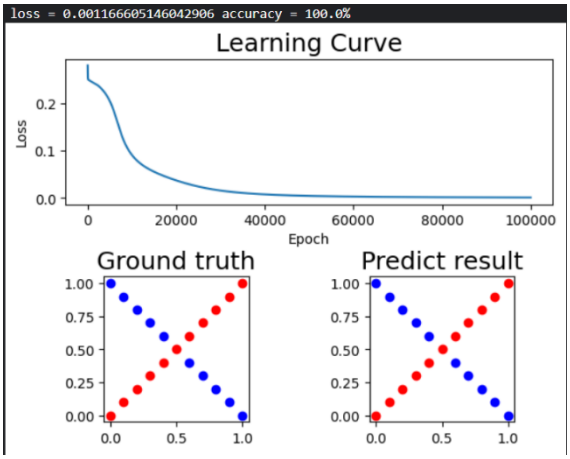
D. Anything you want to share

To verify that XOR data cannot be solved using a linear approach, I tested two networks: one with 10 hidden layers, each with 10 hidden dimensions but without activation functions, and the other with no hidden layers, i.e., only input and output layers, but with a sigmoid activation function. Below are the results.

complex , without activation functions:



Simple, with activation functions:



From this, it is evident that breaking linearity is important for handling XOR data.

5. Extra

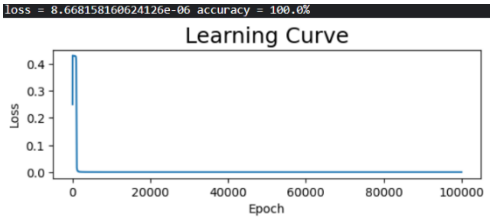
A. Implement different optimizers

In addition to SGD, I also implemented Adam and RMSprop optimizers.

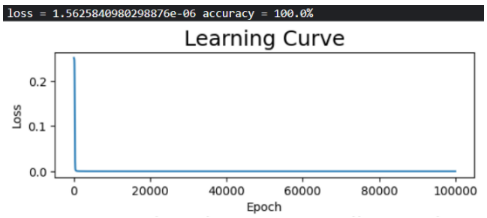
Adam:

```python
class Adam_optimizer:
    def __init__(self, beta1 = 0.9, beta2 = 0.999, epsilon = 1e-8):
        self.name = "Adam"
        self.m_dW = 0
        self.m_db = 0
        self.v_dW = 0
        self.v_db = 0
        self.v_dW_hat = epsilon
        self.v_db_hat = epsilon
        self.beta1, self.beta2 = beta1, beta2
        self.epsilon = epsilon
        self.t = 1
        #initial t = 1

    def update(self, lr, W, b, dW, db):
        self.m_dW = self.beta1 * self.m_dW + (1 - self.beta1) * dW
        self.m_db = self.beta1 * self.m_db + (1 - self.beta1) * db
        # using beta1 to calculate Momentum
        self.v_dW = self.beta2 * self.v_dW + (1 - self.beta2) * (dW ** 2)
        self.v_db = self.beta2 * self.v_db + (1 - self.beta2) * db
        # using beta2 to calculate RMS

        m_dW_hat = self.m_dW / (1 - self.beta1 ** self.t)
        m_db_hat = self.m_db / (1 - self.beta1 ** self.t)
        self.v_dW_hat = np.maximum(self.v_dW / (1 - self.beta2 ** self.t), self.v_d
        self.v_db_hat = np.maximum(self.v_db / (1 - self.beta2 ** self.t), self.v_d
        #new m, v according to time t

        self.W, self.b = W, b
        self.W -= lr * (m_dW_hat / (np.sqrt(self.v_dW_hat) + self.epsilon))
        self.b -= lr * (m_db_hat / (np.sqrt(self.v_db_hat) + self.epsilon))
        self.t += 1
        #update t

        return self.W, self.b
```

Linear:



XOR:

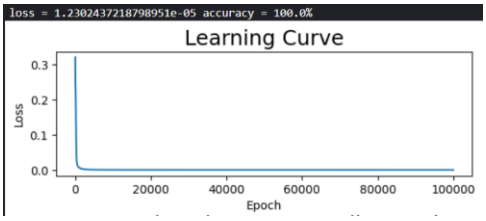


Using Adam allows the learning rate to adjust automatically, also reducing oscillations and achieving a faster convergence rate.

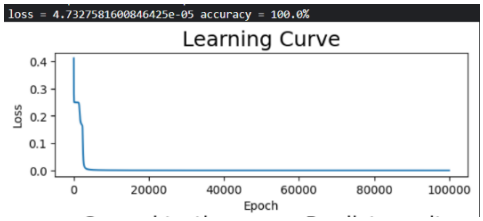
RMSprop:

```python
class RMSprop_optimizer:
    def __init__(self, beta=0.9, epsilon=1e-8):
        self.name = "RMSprop"
        self.beta = beta
        self.epsilon = epsilon
        self.v_dW, self.v_db = epsilon, epsilon

    def update(self, lr, W, b, dW, db):
        self.v_dW = np.maximum(self.beta * self.v_dW + (1 - self.beta) * (dW ** 2), self.v_dW)
        self.v_db = np.maximum(self.beta * self.v_db + (1 - self.beta) * db, self.v_db)

        self.W, self.b = W, b
        self.W -= lr * (dW / (np.sqrt(self.v_dW) + self.epsilon))
        self.b -= lr * (db / (np.sqrt(self.v_db) + self.epsilon))
        return self.W, self.b
```

Linear:



XOR:



The performance is similar to Adam, but Adam produces a smoother loss curve compared to RMSprop when training on XOR data.

B. Implement different activation functions

In addition to Sigmoid, I also implemented ReLU, tanh and ELU.

ReLU:

```python
def ReLU(x, derivative = False):
    if not derivative:
        return np.where(x > 0, x, 0)
    else:
        return np.where(x > 0, 1, 0)
```

tanh:

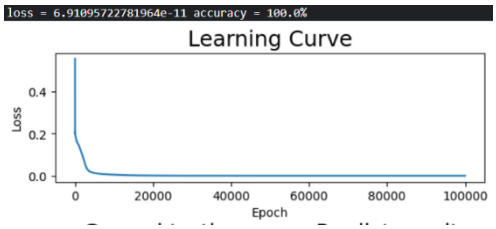
```python
def tanh(x, derivative = False):
    if not derivative:
        return np.tanh(x)
    else:
        return 1 - ( np.tanh(x)**2 )
```

ELU:

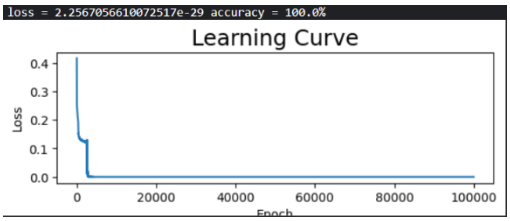
```python
def ELU(x, alpha=1.0, derivative = False):
    if not derivative:
        return np.where(x <= 0, alpha * (np.exp(x) - 1), x)
    else:
        return np.where(x < 0, alpha * np.exp(x), 1)
```

ReLU:

Linear:

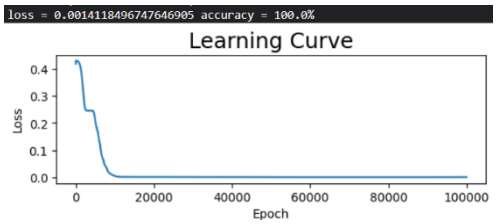


XOR:

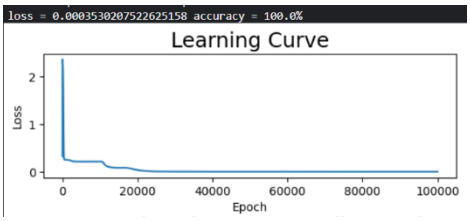


tanh:

Linear:

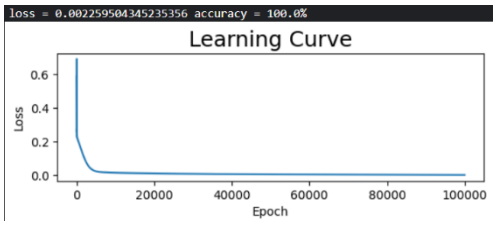


XOR:

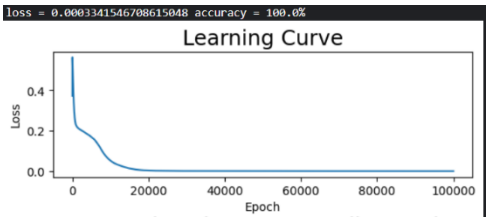


ELU:

Linear:



XOR:

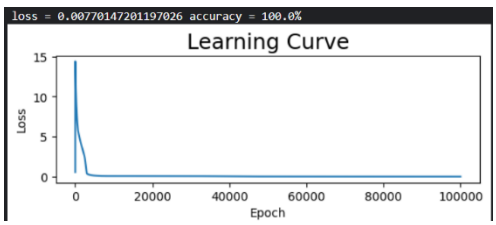


Different activation functions require different parameters settings to learn.

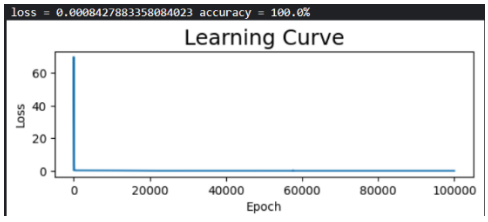
C. Implement convolutional layers

I implemented 1D convolution (conv1D), which adjusts input and output through filter size, stride, and padding settings. In this layer, only the parameters in the filter and the scalar bias are updated through gradient descent. Other hyperparameters and functions are similar to those in fully connected layers.

Linear:



XOR:



```python
class Conv1d_Layer:
    def __init__(self, input_dim = 2, filter_size = 2, stride = 1, padding = 0, activation = "Sigmoid", alpha = 0.1):
        self.input_dim = input_dim
        self.filter_size = filter_size
        self.stride = stride
        self.padding = padding
        self.activation = activation
        self.alpha = alpha

        self.filter = np.random.randn(self.filter_size) * np.sqrt(2 / self.input_dim)
        # He initilize
        #self.bias = np.random.randn(1)
        self.bias = np.random.randn(1) * 0.01

        # create filters and biases for CNN
        #bias just a scalar in this case
        self.A_prev = None
        self.A_prev_padded = None
        self.Z = None
        self.A = None
        self.dW = None
        self.db = None
        self.dA_prev = None
        # for forward pass and backward pass

    def forward(self, A_prev):
        self.A_prev = A_prev
        num_samples = A_prev.shape[0]
        # there may be 2D input if there are multiple layers
        output_length = (self.input_dim - self.filter_size + 2 * self.padding) // self.stride + 1
        #calculate output_length for this layer
        if self.padding > 0:
            #self.A_prev_padded = np.pad(self.A_prev, ((0, 0), (self.padding, self.padding)), mode='constant', constant_values=(1e-8, 1e-8))
            self.A_prev_padded = np.pad(self.A_prev, ((0, 0), (self.padding, self.padding)), mode='constant', constant_values=(0, 0))
        else:
            self.A_prev_padded = self.A_prev
        #apply padding
        #np.pad(array, pad_width, mode='constant', **kwargs)

        self.Z = np.zeros((num_samples, output_length))

        for i in range(output_length):
            start = i * self.stride
            end = start + self.filter_size
            A_prev_slice = self.A_prev_padded[:, start:end]
            self.Z[:, i] = np.dot(A_prev_slice, self.filter) + self.bias


        match self.activation:
            #compute A with Z pass through activation function
            case "Sigmoid":
                self.A = Sigmoid(self.Z, derivative = False)
            case "ReLU":
                self.A = ReLU(self.Z, derivative = False)
            case "tanh":
                self.A = tanh(self.Z, derivative = False)
            case "ELU":
                self.A = ELU(self.Z, self.alpha, derivative = False)
            case _:
                #for no activation function
                self.A = self.Z

        return self.A



    def backward(self, dA):
        A_prev_shape = self.A_prev.shape[0]

        if self.activation == "Sigmoid":
            dZ = dA * Sigmoid(self.A, derivative=True)
        elif self.activation == "ReLU":
            dZ = dA * ReLU(self.A, derivative=True)
        elif self.activation == "tanh":
            dZ = dA * tanh(self.A, derivative=True)
        elif self.activation == "ELU":
            dZ = dA * ELU(self.A, self.alpha, derivative=True)
        else:
            dZ = dA

        num_samples = dZ.shape[0]
        self.dW = np.zeros_like(self.filter)
        self.db = np.sum(dZ)
        self.dA_prev_padded = np.zeros_like(self.A_prev_padded)

        output_length = self.Z.shape[1]

        for i in range(output_length):
            start = i * self.stride
            end = start + self.filter_size
            self.dW += np.sum(self.A_prev_padded[:, start:end] * dZ[:, i][:, np.newaxis], axis=0)
            self.dA_prev_padded[:, start:end] += dZ[:, i][:, np.newaxis] * self.filter

        if self.padding > 0:
            self.dA_prev = self.dA_prev_padded[:, self.padding:-self.padding]
        else:
            self.dA_prev = self.dA_prev_padded

        return self.dA_prev

    def update_parameters(self, lr, optim):
        match optim.name:
            case "SGD":
                self.filter, self.bias = optim.update(lr, self.filter, self.bias, self.dW, self.db)
            case "Adam":
                self.filter, self.bias = optim.update(lr, self.filter, self.bias, self.dW, self.db)
            case "RMSprop":
                self.filter, self.bias = optim.update(lr, self.filter, self.bias, self.dW, self.db)
```